

Tarea Corta 3 – Listas: Caja de Herramientas

Esta tarea consiste en la implementación de un programa que simula una caja de herramientas con diferentes capacidades de peso y volumen y que permite realizar algunas acciones como agregar y borrar. Para eso se requiere la programación y utilización de las siguientes estructuras de datos:

1. Lista con arreglos (*ArrayList*)
2. Lista enlazada simple (*LinkedList*)
3. Lista doblemente enlazada (*DLinkedList*)

Parte 1 – Estructuras de Datos

Implementar las clases *ArrayList*, *LinkedList* y *DLinkedList*. Completar cualquier método que haya quedado pendiente en clases. Estas clases deben implementar todos los métodos de la clase abstracta *List*:

1. **void insert(E element)**
Inserta un nuevo elemento en la posición actual.
2. **void append(E element)**
Inserta un nuevo elemento al final de la lista.
3. **void setElement(E element)**
Sustituye el elemento en la posición actual de la lista.
4. **E getElement()**
Retorna el elemento encontrado en la posición actual.
5. **E remove()**
Elimina elemento en la posición actual de la lista. Retorna como resultado el elemento borrado.
6. **void clear()**
Elimina todos los elementos de la lista y la deja vacía.
7. **void goToStart()**
Mueve la posición actual al inicio de la lista.
8. **void goToEnd()**
Mueve la posición actual al final de la lista.
9. **void goToPos(int pos)**
Mueve la posición actual a la posición indicada por parámetro.
10. **void next()**
Mueve la posición actual a la siguiente posición. Si la posición se encuentra al final de la lista no hace nada.
11. **void previous()**
Mueve la posición actual a la posición anterior. Si la posición se encuentra al inicio de la lista no hace nada.
12. **bool atStart()**

Retorna verdadero si la posición actual de la lista se encuentra al inicio. De otro modo retorna falso.

13. **bool atEnd()**

Retorna verdadero si la posición actual de la lista se encuentra al final. De otro modo retorna falso.

14. **int getPos()**

Retorna un entero con la posición actual dentro de la lista.

15. **int getSize()**

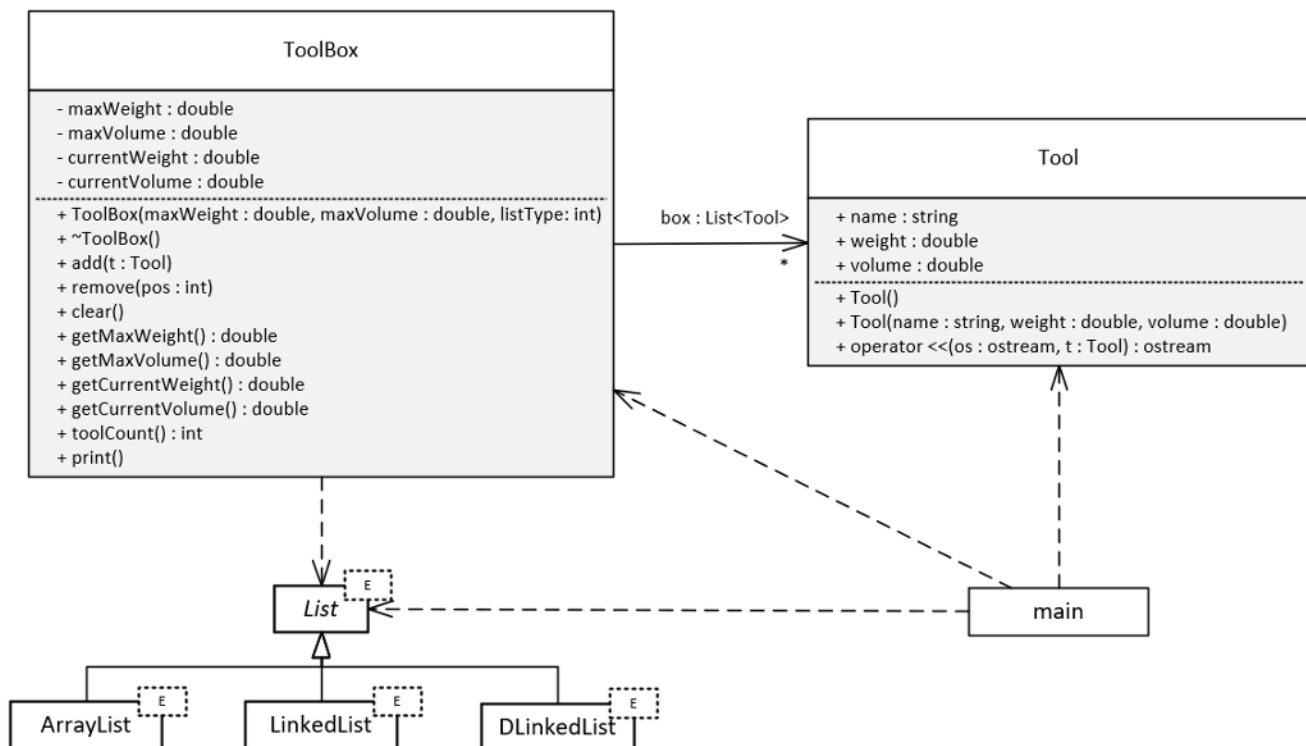
Retorna el tamaño de la lista.

16. **void print()**

Imprime los contenidos de la lista en pantalla.

Parte 2 – Lógica

Para modelar las funcionalidades del programa se van a utilizar dos clases de lógica para representar la caja (*ToolBox*) y las herramientas que contiene (*Tool*).



Tool

Esta clase representa las herramientas a almacenar en la caja.

Tiene tres atributos para las herramientas: nombre, peso y volumen. Nótese que estos atributos son públicos para facilitar su utilización.

La clase debe tener un constructor vacío para que se pueda utilizar con *ArrayList* y crear nodos centinela para las listas enlazadas.

Debe implementar el operador de extracción (<<) para poder utilizar el método *print* de las listas. A continuación, se muestra un ejemplo de dicho método, que muestra el nombre de la herramienta junto con el peso y el volumen.

```
friend ostream& operator <<(ostream & os, const Tool & tool) {  
    os << tool.name << " (w=" << tool.weight << ", v=" << tool.volume << ")";  
    return os;  
}
```

En esta clase no es necesario hacer nada adicional con respecto a la regla de tres de C++.

TooBox

Esta clase representa la caja de herramientas.

Tiene atributos privados para representar el peso máximo en la caja, volumen máximo, el peso y el volumen actuales de sus contenidos.

Tiene una relación de asociación con la clase *Tool*, lo que significa que también tiene un atributo privado que es un puntero polimórfico tipo *List* para almacenar las herramientas.

El constructor debe encargarse de inicializar los atributos y, dependiendo del valor en el parámetro *listType*, debe instanciar un *ArrayList*, *LinkedList* o *DLinkedList* en memoria dinámica para almacenar las herramientas.

El destructor debe encargarse de eliminar la lista de memoria dinámica.

En esta clase sí es necesario eliminar el constructor de copia y el operador de asignación.

El método *add* se encarga de agregar una herramienta a la caja. Si el peso actual de la caja más el peso de la herramienta a almacenar es mayor que el peso máximo permitido en la caja debe lanzar una excepción con un mensaje descriptivo. Si el volumen actual de la caja más el volumen de la herramienta a almacenar es mayor que el volumen máximo permitido en la caja debe lanzar una excepción con un mensaje descriptivo. De otro modo, debe actualizar el peso y volumen actuales con los datos de la herramienta y agregarla a la lista.

El método *remove* recibe la posición de la herramienta a borrar. Si la caja se encuentra vacía debe lanzar una excepción. De otro modo se elimina la herramienta de la lista y se resta al peso y volumen actuales de la caja.

El método *clear* limpia por completo la caja de herramientas.

Los métodos *get* retornan el peso y volumen máximos, peso y volumen actuales de la caja.

El método *toolCount* retorna la cantidad de herramientas que hay en la caja.

El método *print* imprime los contenidos de la caja y demás datos de la caja en pantalla.

Parte 3 – Programa principal

El programa principal debe contar con una lista de herramientas disponibles y una caja de herramientas para administrar.

Al iniciar, el programa debe preguntar al usuario el tipo de listas que desea utilizar, tanto para la lista de herramientas disponibles como para la caja.

Choose the type of list to use:

1. ArrayList
2. LinkedList
3. DLinkedList

Posteriormente, el programa pregunta al usuario el tamaño de la caja de herramientas a utilizar (pequeña, mediana o grande).

Choose the size of the toolbox:

1. Small (max weight = 2, max volume = 10)
2. Medium (max weight = 4, max volume = 14)
3. Large (max weight = 6, max volume = 20)

Luego, el programa se mantiene en un ciclo principal que muestra los contenidos y datos de la caja (con el método *print*) y las operaciones a realizar hasta que el usuario elija la opción para salir.

Toolbox contents:

Weight: 0/4

Volume: 0/14

Choose toolbox operation:

1. Add tool
2. Remove tool
3. Clear toolbox
0. Exit

En la operación para agregar una herramienta a la caja, se muestran 12 herramientas diferentes y el usuario elige una para agregar a la caja.

Choose tool to add:

1. Phillips screwdriver (w=0.25, v=1.5)
2. Flat screwdriver (w=0.25, v=1.5)
3. Hammer (w=1, v=3.5)
4. French wrench (w=0.75, v=3)
5. Hacksaw (w=0.5, v=3)
6. Scissors (w=0.25, v=2.2)
7. Cutter (w=0.25, v=1.5)
8. Pliers (w=0.5, v=1.8)
9. Pincers (w=0.25, v=2)
10. Drill (w=2, v=4)
11. Locking pliers (w=0.5, v=2.5)
12. Measuring tape (w=0.5, v=1.5)

Si la caja puede almacenar la herramienta, entonces se muestra un mensaje indicando que se agregó. De otro modo se muestra el mensaje de error que indica lo que sucedió (peso o volumen excedido). En cualquier caso se continúa con el ciclo principal.

Toolbox contents:

1. Phillips screwdriver (w=0.25, v=1.5)
2. Flat screwdriver (w=0.25, v=1.5)
3. Hammer (w=1, v=3.5)
4. French wrench (w=0.75, v=3)
5. Hacksaw (w=0.5, v=3)

Weight: 2.75/4

Volume: 12.5/14

Choose toolbox operation:

1. Add tool
 2. Remove tool
 3. Clear toolbox
 0. Exit
- 1

Choose tool to add:

1. Phillips screwdriver (w=0.25, v=1.5)
 2. Flat screwdriver (w=0.25, v=1.5)
 3. Hammer (w=1, v=3.5)
 4. French wrench (w=0.75, v=3)
 5. Hacksaw (w=0.5, v=3)
 6. Scissors (w=0.25, v=2.2)
 7. Cutter (w=0.25, v=1.5)
 8. Pliers (w=0.5, v=1.8)
 9. Pincers (w=0.25, v=2)
 10. Drill (w=2, v=4)
 11. Locking pliers (w=0.5, v=2.5)
 12. Measuring tape (w=0.5, v=1.5)
- 6

Error: The toolbox volume limit has been exceeded.

En la opción para borrar una herramienta de la caja, se muestra una lista con las herramientas dentro de la caja y el usuario elige cuál quiere eliminar y se realiza la operación. Luego se continúa con el ciclo principal.

```
Choose toolbox operation:
1. Add tool
2. Remove tool
3. Clear toolbox
0. Exit
2

Toolbox contents:
    1. Phillips screwdriver (w=0.25, v=1.5)
    2. Flat screwdriver (w=0.25, v=1.5)
    3. Hammer (w=1, v=3.5)
    4. Hacksaw (w=0.5, v=3)
Weight: 2/2
Volume: 9.5/10
Choose tool to remove:
3

Toolbox contents:
    1. Phillips screwdriver (w=0.25, v=1.5)
    2. Flat screwdriver (w=0.25, v=1.5)
    3. Hacksaw (w=0.5, v=3)
Weight: 1/2
Volume: 6/10
```

En caso de que se trate de borrar una herramienta cuando la caja esté vacía se debe mostrar un mensaje indicando que ya se encuentra vacía.

```
Choose toolbox operation:
1. Add tool
2. Remove tool
3. Clear toolbox
0. Exit
2
Toolbox is empty!
```

La opción para limpiar la caja debe regresar la caja a su estado inicial.

```
Choose toolbox operation:
1. Add tool
2. Remove tool
3. Clear toolbox
0. Exit
3

Toolbox contents:
Weight: 0/2
Volume: 0/10
```

La opción para salir termina el programa.

Otros detalles del programa principal

1. La lista con herramientas disponibles debe cargarse en el programa principal. Debe usarse el tipo de lista que el usuario elige al inicio del programa, por lo que debe usarse un puntero polimórfico.
2. Se recomienda crear el objeto de la caja de herramientas en memoria dinámica. Internamente, la caja de herramientas también debe usar el tipo de lista que el usuario haya elegido.
3. El programa debe utilizar manejo de errores con la estructura *try/catch* para detectar cualquier situación anómala. Incluyendo los errores que puedan provocar los métodos de la caja.
4. El programa debe rechazar cualquier entrada no válida escrita por el usuario y volverla a solicitar.
5. Los menús de la interfaz de usuario deben usar opciones numéricas como las mostradas.
6. Se recomienda utilizar solamente la función *getline* para leer las opciones del usuario.
7. El programa, al finalizar, debe eliminar los objetos que se hayan creado en memoria dinámica.
8. Divida el programa principal en subrutinas.
9. El peso y volumen máximo de los tres tamaños de las cajas, las herramientas de la lista y sus características pueden variarse según se desee. Pero deben ser al menos tres tamaños de caja y al menos doce herramientas.
10. No es obligatorio que la interfaz de usuario esté en inglés.

¡Buena suerte!

Entrega: Sección de evaluaciones del TEC-Digital.

Formato: Archivo comprimido ZIP con el proyecto completo de Visual Studio. Corrobore que se incluyen todos los archivos necesarios para compilar y ejecutar el proyecto. Recuerde limpiar los archivos de compilación del proyecto antes de comprimir y no incluir la carpeta oculta “.vs” dentro del ZIP.